

Real-Time Embedded Systems Standard Final Project Report

Phil Orlando
Graduate Student, Department of Electrical Engineering
University of Colorado Boulder

All source code used for this assignment is available at <https://github.com/Pmorlando/RTES-Orlando/tree/main/Finalproj> and if issues downloading or viewing are encountered, please contact me. I am not the most experienced with Github but happy to learn of mistakes I am making.

I chose to complete the standard project where I am capturing images of an analog clock that does not skip a second or catch blur while the second hand is moving. Additionally, the second service I am running on the same core is sharpening the captured picture of the clock and writing both images to storage. For my design in figure (1), I first wanted to think of a way to not have the image repeat itself and not capture the same position of the second hand. My plan was to use the sequencer code and have sequence 1's function be reading an image from the camera via V4L2 code from the class Github and calculate the difference between the current frame and the previous frame. I imagined that there would be a measurable difference between frames and at 30 FPS there could be a few frames that would capture the second hand movement. Therefore, my plan was to compare the last five frames and look for any of the last five frames to have a spike in frame difference percentage above some threshold and then have the last difference capture percentage value be below the threshold. This would allow the second hand to stop moving and then I would use that frame as the per second capture.

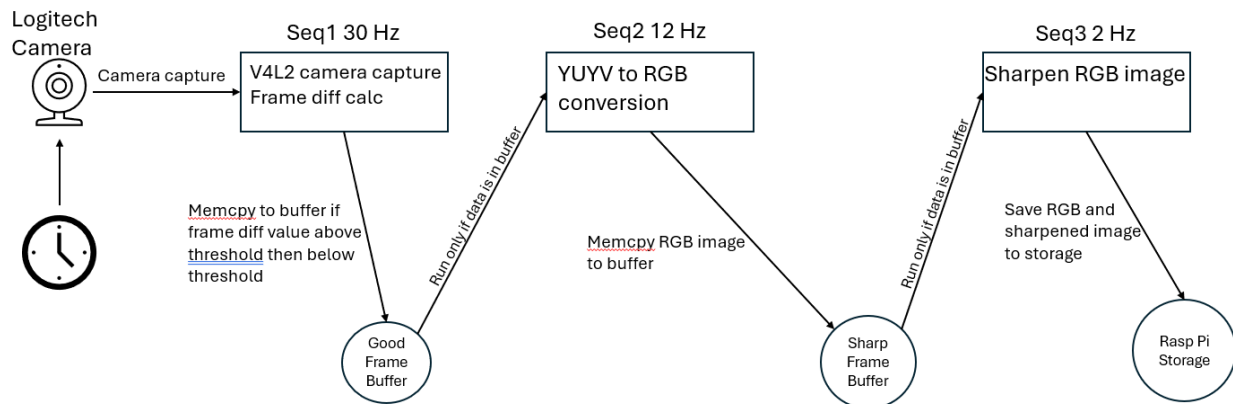


Figure (1) Block diagram.

I began by setting up my laptop with the analog clock and the webcam to test if my camera was running at 30 Hz when I hardcoded in the camera initialization and proved this by capturing 900 frames and average the FPS [1]. After a few rounds of testing my webcam was averaging just over 30 FPS so I knew having sequencer 1 at 30 Hz is a realistic place to start.

```
philorlando@Pi4: ~/Documents/RTES-Orlando/Finalproj
capture.c - /home/phi...
File Edit Tabs Help
allocated buffer 0
allocated buffer 1
allocated buffer 2
allocated buffer 3
allocated buffer 4
allocated buffer 5
philorlando@Pi4:~/Documents/RTES-Orlando/Finalproj $ journalctl -f | grep capture
Jul 21 21:59:12 Pi4 capture[2327]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:12 Pi4 capture[2327]: frame 889: frame time 32.068, FPS = 31.184
Jul 21 21:59:12 Pi4 capture[2327]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:12 Pi4 capture[2327]: frame 890: frame time 35.993, FPS = 27.783
Jul 21 21:59:12 Pi4 capture[2327]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:12 Pi4 capture[2327]: frame 891: frame time 31.975, FPS = 31.275
Jul 21 21:59:12 Pi4 capture[2327]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:12 Pi4 capture[2327]: frame 892: frame time 32.054, FPS = 31.197
Jul 21 21:59:12 Pi4 capture[2327]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:12 Pi4 capture[2327]: cap test n=892 acg FPS = 30.04
^C
philorlando@Pi4:~/Documents/RTES-Orlando/Finalproj $ ./capture
FORCING FORMAT
allocated buffer 0
allocated buffer 1
allocated buffer 2
allocated buffer 3
allocated buffer 4
allocated buffer 5
philorlando@Pi4:~/Documents/RTES-Orlando/Finalproj $ journalctl -f | grep capture
Jul 21 21:59:58 Pi4 capture[2330]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:58 Pi4 capture[2330]: frame 889: frame time 36.012, FPS = 27.768
Jul 21 21:59:58 Pi4 capture[2330]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:58 Pi4 capture[2330]: frame 890: frame time 31.991, FPS = 31.259
Jul 21 21:59:58 Pi4 capture[2330]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:58 Pi4 capture[2330]: frame 891: frame time 32.008, FPS = 31.242
Jul 21 21:59:58 Pi4 capture[2330]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:58 Pi4 capture[2330]: frame 892: frame time 35.995, FPS = 27.781
Jul 21 21:59:58 Pi4 capture[2330]: time_error.tv_sec=0, time_error.tv_nsec=0
Jul 21 21:59:58 Pi4 capture[2330]: cap test n=892 acg FPS = 30.03
^C
philorlando@Pi4:~/Documents/RTES-Orlando/Finalproj $
```

Figure (2) Camera speed test.

Next, I began testing the difference capture code to see if ever with such a small second hand, that a meaningful difference measurement could be observed. In my testing I was able to observe this, and the screenshot shows that the difference between frames returns to near zero for the same frames as expected.

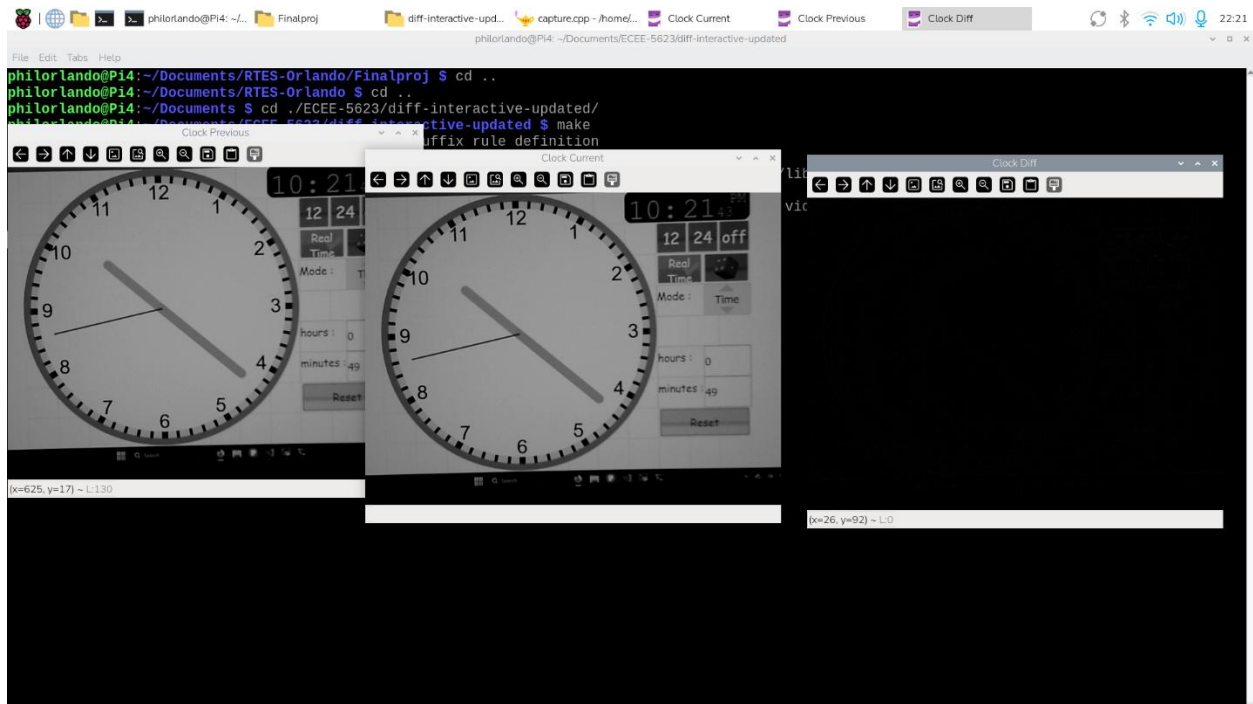
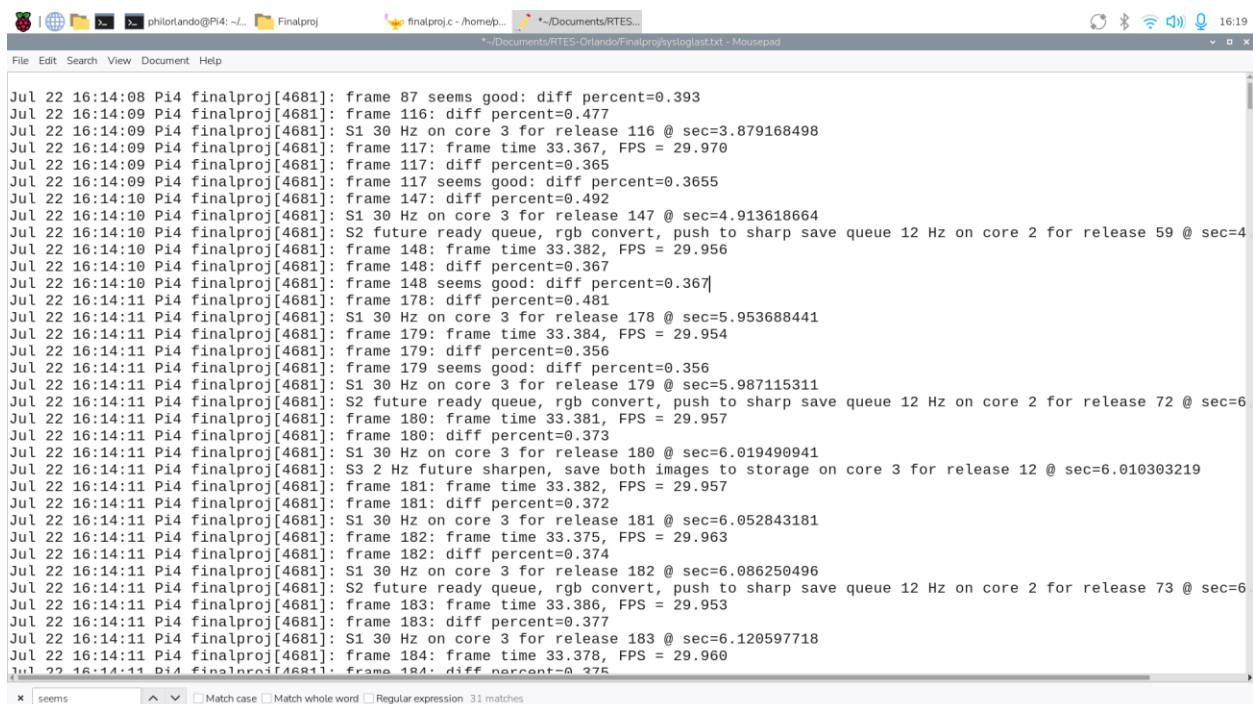


Figure (3) Frame difference testing.

I then combined the Sequencer code, V4L2 capture code, and the diff capture to begin testing the feasibility of my initial design and trigger for if a captured frame is the one I want to save. I set up the syslog outputs to record every frame difference measurement and a trigger if any of the last five frame difference measurements were above zero and the last measurement was zero. I realized that after the first 45 or so frames, the camera initialized and adjusted to the lighting. After it was adjusted, there was a baseline difference between frames of around 0.39. However, I saw about every 30 frames an increase in the frame difference to around 0.48. To correct the camera adjustment delay, the trigger if there is a spike in the frame difference will not output for the first 60 frames to ensure the camera is ready and producing clear images. The above average frame difference measurement for most of the time was a singular increase between two average values indicating the camera captured before and after the second hand moved. Although some instances had two frame difference measurements that must have captured the secondhand movement and before it had settled. This result reinforced my theory to not just keep the frame after an increase in frame difference measurement because I would have stored a still moving second hand. This also proved that the frame difference measurements and capture were working properly. Once I adjusted the threshold for triggering a change in frame difference to 0.42, I was seeing the syslog trigger for if one of the last five frame difference values were above the threshold and the most recent was below the threshold. Though, the spike in frame difference values were only singular or maybe two and the trigger would be showing for multiple frames after the second hand had settled. I then adjusted the detection for the frame difference value to look at the last two values and triggering when the most recent is below the threshold and the second is above the threshold.



```
Jul 22 16:14:08 P14 finalproj[4681]: frame 87 seems good: diff percent=0.393
Jul 22 16:14:09 P14 finalproj[4681]: frame 116: diff percent=0.477
Jul 22 16:14:09 P14 finalproj[4681]: S1 30 Hz on core 3 for release 116 @ sec=3.879168498
Jul 22 16:14:09 P14 finalproj[4681]: frame 117: frame time 33.367, FPS = 29.979
Jul 22 16:14:09 P14 finalproj[4681]: frame 117: diff percent=0.365
Jul 22 16:14:09 P14 finalproj[4681]: frame 117 seems good: diff percent=0.3655
Jul 22 16:14:10 P14 finalproj[4681]: frame 147: diff percent=0.492
Jul 22 16:14:10 P14 finalproj[4681]: S1 30 Hz on core 3 for release 147 @ sec=4.913618664
Jul 22 16:14:10 P14 finalproj[4681]: S2 future ready queue, rgb convert, push to sharp save queue 12 Hz on core 2 for release 59 @ sec=4
Jul 22 16:14:10 P14 finalproj[4681]: frame 148: frame time 33.382, FPS = 29.956
Jul 22 16:14:10 P14 finalproj[4681]: frame 148: diff percent=0.367
Jul 22 16:14:10 P14 finalproj[4681]: frame 148 seems good: diff percent=0.367
Jul 22 16:14:11 P14 finalproj[4681]: frame 178: diff percent=0.481
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 178 @ sec=5.953688441
Jul 22 16:14:11 P14 finalproj[4681]: frame 179: frame time 33.384, FPS = 29.954
Jul 22 16:14:11 P14 finalproj[4681]: frame 179: diff percent=0.356
Jul 22 16:14:11 P14 finalproj[4681]: frame 179 seems good: diff percent=0.356
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 179 @ sec=5.987115311
Jul 22 16:14:11 P14 finalproj[4681]: S2 future ready queue, rgb convert, push to sharp save queue 12 Hz on core 2 for release 72 @ sec=6
Jul 22 16:14:11 P14 finalproj[4681]: frame 180: frame time 33.381, FPS = 29.957
Jul 22 16:14:11 P14 finalproj[4681]: frame 180: diff percent=0.373
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 180 @ sec=6.019490941
Jul 22 16:14:11 P14 finalproj[4681]: S3 2 Hz future sharpen, save both images to storage on core 3 for release 12 @ sec=6.010303219
Jul 22 16:14:11 P14 finalproj[4681]: frame 181: frame time 33.382, FPS = 29.957
Jul 22 16:14:11 P14 finalproj[4681]: frame 181: diff percent=0.372
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 181 @ sec=6.052843181
Jul 22 16:14:11 P14 finalproj[4681]: frame 182: frame time 33.375, FPS = 29.963
Jul 22 16:14:11 P14 finalproj[4681]: frame 182: diff percent=0.374
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 182 @ sec=6.086250496
Jul 22 16:14:11 P14 finalproj[4681]: S2 future ready queue, rgb convert, push to sharp save queue 12 Hz on core 2 for release 73 @ sec=6
Jul 22 16:14:11 P14 finalproj[4681]: frame 183: frame time 33.386, FPS = 29.953
Jul 22 16:14:11 P14 finalproj[4681]: frame 183: diff percent=0.377
Jul 22 16:14:11 P14 finalproj[4681]: S1 30 Hz on core 3 for release 183 @ sec=6.120597718
Jul 22 16:14:11 P14 finalproj[4681]: frame 184: frame time 33.378, FPS = 29.960
Jul 22 16:14:11 P14 finalproj[4681]: frame 184: diff percent=0.375
```

seems 31 matches

Figure (4) Frame difference triggered with “seems good”.

I then moved to implementing a method for the second and third sequences. I was not sure at what frequency the second or third sequence should be running at and analyzed what their purpose would be before deciding. I decided that the second sequence would be converting the V4L2 images from YUYV to RGB and that the third sequence would be taking the RGB frames, sharpen the frame and store the RGB and sharpened frame. These would only be needed once per second but to avoid having all the image processing in one sequence and reducing the context switching. I then decided to go with 12 Hz for the second sequence and 2 Hz for the third sequence.

My solution to avoid the issue of unbounded priority inversion between the sequences was to add buffers from sequences 1 to 2 and sequences 2 to 3 [2]. This avoids the shared memory between three services of different priority and prevents unbounded priority inversion. The two buffers are the good frame buffer for if sequence 1 finds a frame worth saving and sequence 2 will copy from this, and sharp frame buffer for sequence 2 to save the RGB frames into and sequence 3 to copy the frames ready to be sharpened out of. To preserve the data copying to and from the buffers, I used mutexes with priority inheritance to protect the reading and writing to the buffers [3]. Although with sequence 2 working at 12 Hz and sequence 1 writing to the buffer at 1 Hz, there should be no time where they are accessing the buffer at the same time but to avoid future headaches, I added this in with the buffers. The same concept was used for the sharp buffer where it should only be written to and read once per second and should not be accessed by both sequences at the same time.

After completing the coding for sequence 2 and 3, I tested for around 30 seconds and found success in getting approximately 30 RGB and 30 sharpened images of the analog clock without any doubles or blur on the second hand. As you can see in figure 5, I made a mistake in sizing the RGB frame and it carried over to the sharpened image [4].

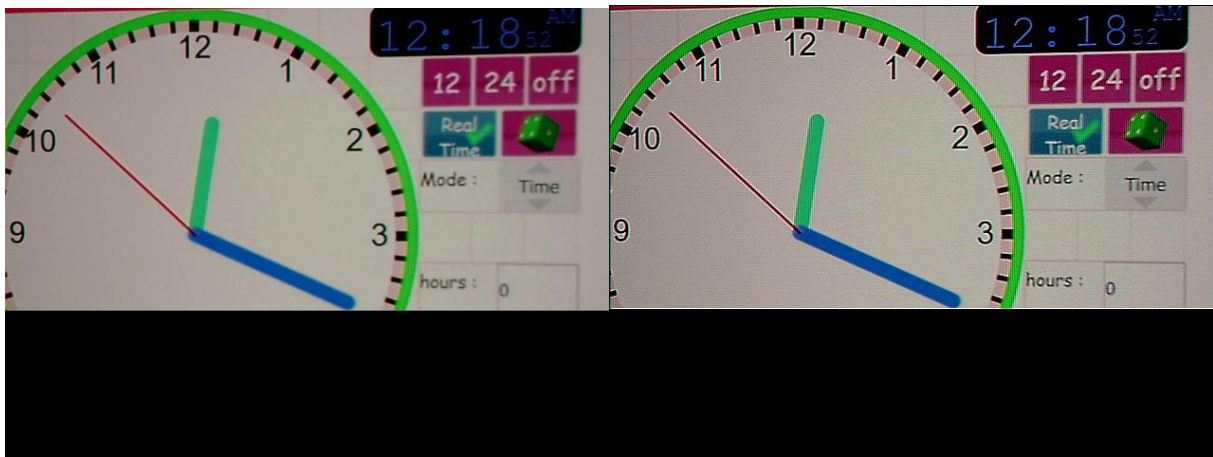


Figure (5) First test results for RGB and Sharpened image.

After correcting the mistake of the RGB frame size, I ran a correct successful 30 second test as seen in figure 6.



Figure (6) Correct sized RGB and Sharpened images.

I then ran ten tests of approximately one minute long to record the WCET of each sequence to analyze in cheddar. The results can be seen in Table 1, and the highlighted values are the WCET I used for the cheddar analysis. The reason for ignoring the highest sequence 3 WCET was because my laptop went to sleep during the test and waking the screen back up and opening caused multiple frames in a row to be flagged as good frames and to get sharpened and saved. Therefore, I do not believe those times are a fair representation of the runtime of sequence 3 and I have left them out.

Sequence 1	Sequence 2	Sequence 3
0.016056037 sec	0.021112908 sec	0.059223556 sec
0.012142445 sec	0.023192148 sec	0.060228277 sec
0.012137185 sec	0.021174797 sec	0.081461092 sec
0.013791074 sec	0.023432167 sec	0.042275537 sec
0.012113260 sec	0.021096314 sec	0.057562222 sec
0.013808666 sec	0.022313240 sec	0.042037019 sec
0.012109741 sec	0.023222074 sec	0.041963797 sec
0.013149612 sec	0.022718796 sec	0.041956074 sec
0.013821926 sec	0.021447593 sec	0.042032555 sec
0.013169815 sec	0.021024574 sec	0.041998963 sec

Table (1) WCET for ten one-minute test runs.

Using the Utility formula $U = \frac{c_1}{T_1} + \frac{c_2}{T_2} + \frac{c_3}{T_3}$, with the current frequency for the sequences being 30 Hz, 12 Hz and 2 Hz respectively and the highlighted WCET, I calculated the total utility to be 0.9258. Sequence 1 utilized approximately 48% of the computing time, sequence 2 used 28% and sequence 3 used 12%. The Liu and Leyland Rate Monotonic Least Upper Bound (RM LUB) formula for a rough estimate of utility upper bound for 3 services is 0.7798.

Realizing my utility is over 90% did not initially make sense to me but it was because sequence 2 and 3 will run on average once per second if the capture is happening at the same rate. In a worst-case scenario for sequence 2 and 3, they may run twice over the course of one

second. Consequently, I discovered my unnecessarily high frequency for sequence 2 is making the utility calculation inconsistent to the actual function of my code. I therefore changed the frequency of sequence 2 to 4 Hz to decrease the utilization and more align with the needs of the system. This decreased the utilization of sequence 2 from 28% to around 9% of the computing time and reduced the overall utility to 0.696. I did not want to decrease to a lower frequency so that sequence 2 can have two opportunities to convert good frames to RGB for sequence 3. Additionally, the utility is safely below the Liu and Leyland RM LUB.

To simulate this in cheddar I needed all values in whole numbers, so I converted all values in terms of milliseconds and rounded them to the nearest whole integer. The results for the Cheddar simulation over the longest period can be seen in figure 7 and 8. The feasibility tests pass and the utility is below the RM LUB proving this task set is schedulable.

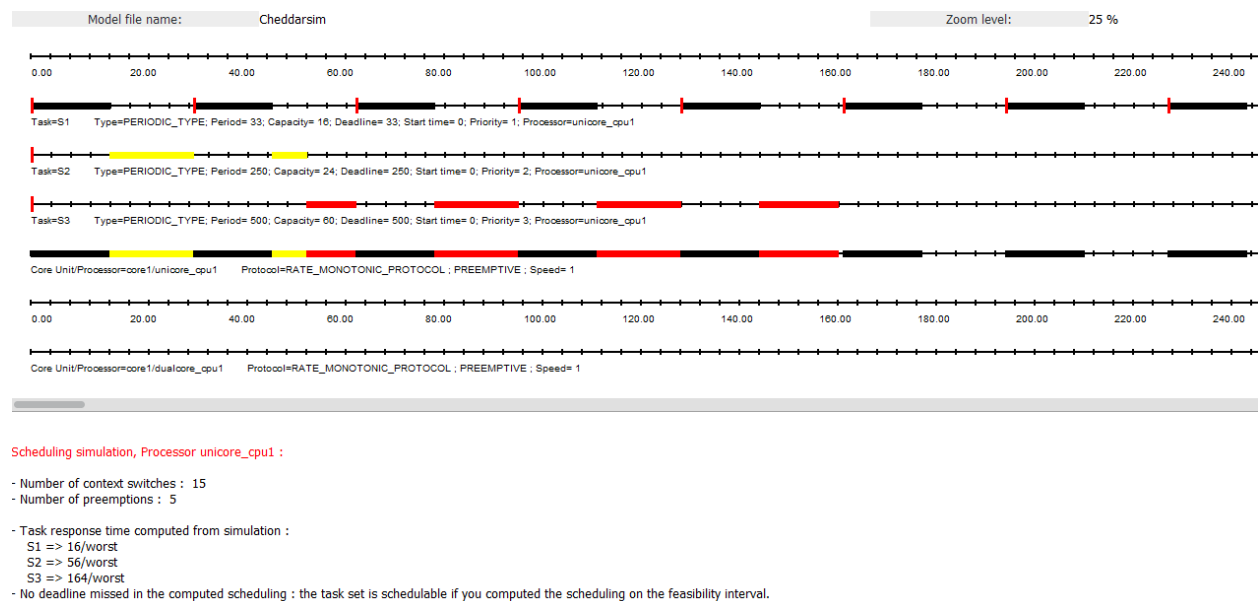


Figure (7) Cheddar simulation over first quarter second.

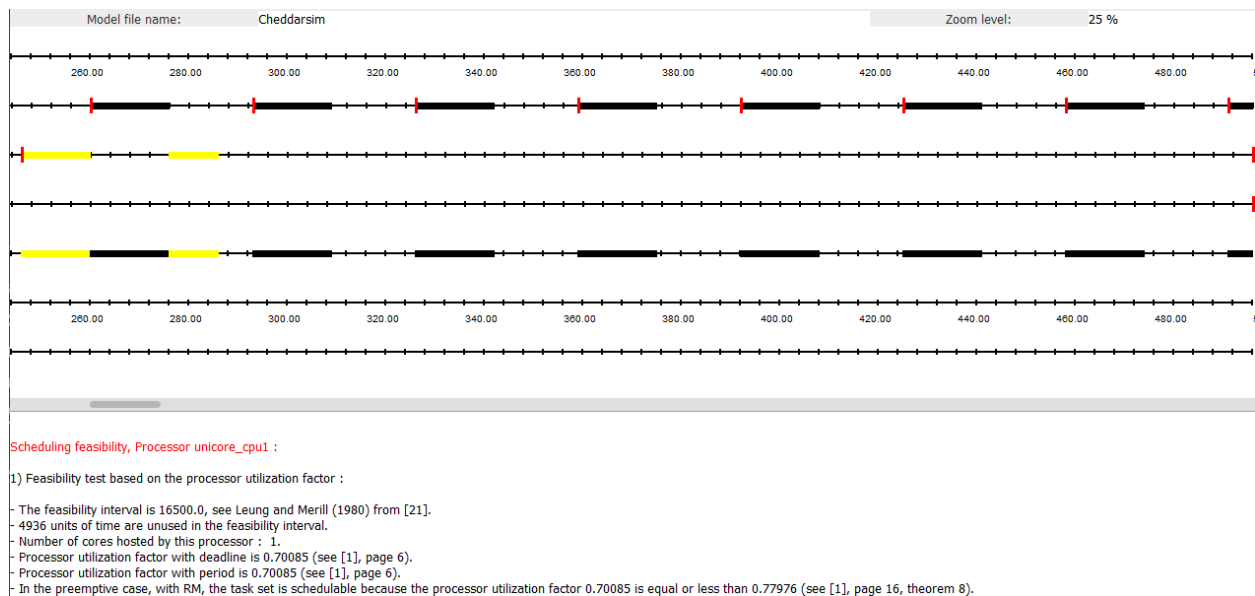


Figure (8) Cheddar simulation over second quarter of a second.

In terms of safety margin, I think a 30% margin putting the total utility around 0.9 is a very large overestimate and in this case with a 1 Hz clock change is not likely to run into an issue that would put the utilization near the 0.9 safety margin.

During final testing, I was able to capture just over 1800 frames with no missing seconds and no blur on the second hand in any of the images. The naming nomenclature I used for naming each frame uses the capture frame for the saved image and subsequent RGB and sharpened image. It saves the file name with four digits. Due to the test running for over 30 minutes, the frames captured exceed four digits and are over five digits for the later frames. Encoding the frames into a video with different digits in the file names had an incorrect order and the video, although showing singular capture per second and no secondhand blur, was not in the correct order [5]. I then adjusted the naming nomenclature to include five digits and ran another test. This successfully capture over 1800 blur free frames with no repeat seconds. I saved this example video and with muscle memory deleted all the single frames which included all the sharpened images, so I did not create a video with the sharpened images. To capture both the RGB and sharpened images I ran another test and noticed the threshold value was higher for this run so after adjusting that was able to capture 1800+ RGB images, sharpened images and make two videos encoding each frame and have linked in my Github repository.

In conclusion, my real time system design is under the RM LUB in utility by a fair margin and can perform the 1 Hz clock synchronization. In regard to the system design, building the system with redundancy and overprotection early on to avoid captures with a blurry second hand helped me complete this project with plenty of utility to spare. By adding extra protection with the mutexes, separate buffers and oversampling sequences 2 and 3, this kept the system from unbounded priority inversion. Throughout the system design I always aimed to overbuild each step along the way to avoid the need for a redesign after testing. I am happy to say this

system can meet the 1 Hz requirements while running another service in the form of image sharpening on the same core.

```
ph1orlando@Pi4:~/Documents/RTES-Orlando/Finalproj/frames $ ffmpeg -f image2 -pattern_type glob -i 'Sharp*.ppm' -vcodec mpeg4 timelapse4.mp4
ffmpeg version 7.1.5-0+deb13u1+rpt1 Copyright (c) 2000-2026 the FFmpeg developers
  built with gcc 14 (Debian 14.2.0-19)
  configuration: --prefix=/usr --extra-version=0+deb13u1+rpt1 --toolchain=hardened --incdir=/usr/include/aarch64-linux-gnu --enable-gpl --disable-str
  able-libass --enable-libb52b --enable-libcdio --enable-libcodecs2 --enable-libdav1d --enable-libflite --enable-libfontconfig --enable-libfreetype --en
  --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-librubberband --enable-libshine --enable-li
  e-libvidstab --enable-libvorbis --enable-libvpx --enable-libwebp --enable-libx265 --enable-libxml2 --enable-libxvid --enable-libzimg --enable-openal
  4-linux-gnu --arch=arm64 --enable-neon --enable-v4l2-request --enable-libudev --enable-epoxy --enable-libdc1394 --enable-libdrm --enable-libiec61883
  --enable-libcaca --enable-libdav1d --enable-libdav1d --enable-libjack --enable-libpulse --enable-librabbitmq --enable-librist --enable-librt --e
  e-v2 --enable-sand --enable-sdl2 --enable-libplacebo --enable-librav1e --enable-pocketsphinx --enable-librav1e --enable-libx264 --enable-shared
  libavutil 59. 39.100 / 59. 39.100
  libavcodec 61. 19.101 / 61. 19.101
  libavformat 61. 7.103 / 61. 7.103
  libavdevice 61. 3.100 / 61. 3.100
  libavfilter 10. 5.100 / 10. 5.100
  libswscale  6. 3.100 /  6. 3.100
  libswresample 5. 3.100 /  5. 3.100
  libpostproc 58. 3.100 / 58. 3.100
Input #0, image2, from 'Sharp*.ppm':
  Duration: 00:01:13.24, start: 0.000000, bitrate: N/A
  Stream #0:0: Video: ppm, rgb24, 640x480, 25 fps, 25 tbr, 25 tbn
Stream mapping:
  Stream #0:0 -> #0:0 (ppm (native) -> mpeg4 (native))
Press [q] to stop, [?] for help
Output #0, mp4, to 'timelapse4.mp4':
  Metadata:
    encoder      : Lavf61.7.103
  Stream #0:0: Video: mpeg4 (mp4v / 0x7634706D), yuv420p(tv, progressive), 640x480, q=2-31, 200 kb/s, 25 fps, 12800 tbn
    Metadata:
      encoder      : Lavc61.19.101 mpeg4
    Side data:
      cpb: bitrate max/min/avg: 0/0/200000 buffer size: 0 vbv_delay: N/A
[out#0/mp4 @ 0x556dc6f030] video:2569K1B audio:0K1B subtitle:0K1B other streams:0K1B global headers:0K1B muxing overhead: 0.336151%
frame= 1631 fps= 42 q=31.0 lsize= 2570K1B time=00:01:13.24 bitrate= 288.3Kbits/s speed=1.69x
```

Figure (9) FFMPEG video encoding screenshot.

References:

- [1] <https://stackoverflow.com/questions/26108643/setting-frame-rate-on-logitech-c210-webcam-in-c-on-raspberry-pi-using-v4l2>

- [2] <https://stackoverflow.com/questions/3621956/how-do-i-code-a-simple-integer-circular-buffer-in-c-c>
- [3] <https://www.ibm.com/docs/en/zos/3.2.0?topic=functions-pthread-mutex-init-initialize-mutex-object#ptminit>
- [4] <https://stackoverflow.com/questions/4491649/how-to-convert-yuy2-to-a-bitmap-in-c>
- [5] <https://stackoverflow.com/questions/48646754/making-a-video-from-arbitrary-named-files-using-ffmpeg>

Appendix 1:

Current code cpp file linked [on Github here](#)

All other files including testing images and video, results video which will be added to before the presentation and screenshots used in the report and linked [on Github here](#)